

WM 2026 Prognosemodell

Wie aus 150 Jahren Fussballdaten, ein paar Python-Notebooks, einem statistischen Modell und einer täglichen GitHub-Automatik ein Live-Dashboard wird, das jeden Morgen frische Titel-Wahrscheinlichkeiten für alle 48 Teams ausgibt.

Inhalt

01	Was das Projekt macht und warum	3
02	Architektur im Überblick	4
03	Phase 1 — Tor-Modell mit Dixon-Coles	5
04	Phase 2 — Monte-Carlo-Simulation des Turniers	7
05	Phase 3 — Elo-Engine als Modell-Upgrade	9
06	Phase 4 — Live-Schicht mit TheSportsDB	11
07	Phase 5 — Pro-Spiel-Prognose (1/X/2)	13
08	Phase 6 — Automatik via GitHub Actions	14
09	Phase 7 — Dashboard auf GitHub Pages	16
10	Reproduktion — bau es selbst nach	17
11	Datei-Referenz	19

Was das Projekt macht — und warum so

Wer wird Weltmeister? Exakt sagen lässt sich das nicht. Aber man kann **Wahrscheinlichkeiten** schätzen — und die jeden Tag mit den frischen Resultaten neu rechnen.

Fussball ist ein Sport, in dem auch starke Mannschaften regelmässig verlieren. Eine Punkt-Prognose wie "Argentinien wird Weltmeister" ist deshalb nur in einem von ungefähr sieben Universen richtig. Sinnvoller ist ein **Verteilungs-Modell**: Argentinien hat 14 %, Spanien 11 %, Frankreich 10 % — und so weiter über alle 48 Teams. Diese Verteilung ist das, was das Projekt jeden Morgen ausgibt.

Das Modell arbeitet auf drei Ebenen, die aufeinander aufbauen:

EBENE	WAS	WIE
A	Sieg/Remis/Niederlage pro Einzelspiel (z.B. 47/13/40 %)	Dixon-Coles-Tor-Modell, später ersetzt durch Elo-Engine
B	Exaktes Resultat als Zwischenschritt (2:1, 0:0, 3:2 ...)	Score-Matrix aus Ebene A
C	Wer wird Weltmeister? (z.B. Argentinien 14 %)	Monte-Carlo-Simulation von 10 000 Turnieren

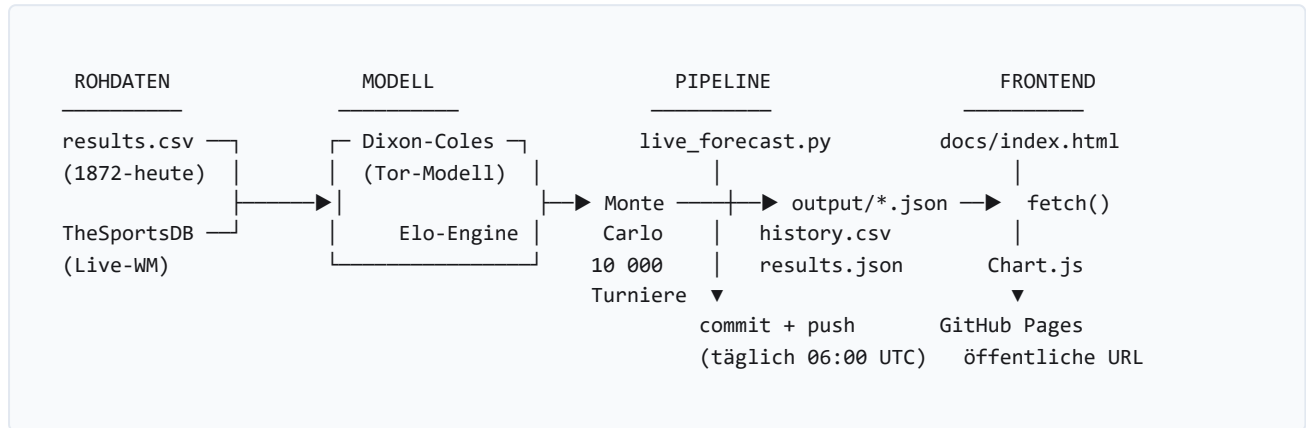
Der rote Faden: *Ebene A liefert die Bausteine, Monte Carlo aggregiert sie zu Ebene C.* Ebene B ist nur Zwischenrechnung für Tordifferenzen (wichtig für Gruppenphasen-Rangfolgen).

WARUM NICHT EINFACH LIGHTGBM?

Klassische ML-Modelle (XGBoost, LightGBM) brauchen viele Features pro Spiel und liefern oft schlechter kalibrierte Wahrscheinlichkeiten als ein gut gewähltes statistisches Modell. Im Backtest auf der WM 2022 schlägt die Elo-Engine die Baseline um 4.4 % RPS-Punkte — eine spätere LightGBM-Iteration steht auf der Roadmap, ist aber kein Selbstzweck.

Architektur im Überblick

Drei Welten greifen ineinander: Daten kommen rein, ein Modell rechnet, Artefakte werden auf GitHub Pages publiziert.



Drei Layer, drei Sprachen

- **Daten-Layer** — CSV mit allen Länderspielen seit 1872 (~ 47 000 Zeilen) als Trainingsgrundlage; TheSportsDB-API für Live-Resultate der laufenden WM.
- **Modell-Layer** — reine Python-Bibliothek in [src/wm_model.py](#). Wird sowohl von Notebooks (zum Experimentieren) als auch vom Automatik-Skript verwendet — eine einzige Quelle der Wahrheit.
- **Publikations-Layer** — GitHub Actions ruft das Skript täglich auf, committet die Ergebnisse zurück ins Repo, GitHub Pages serviert das HTML-Dashboard.

Warum diese Trennung

Notebooks sind super zum Forschen, aber schlecht zum Automatisieren. Deshalb wandert jede Logik, die später in Production läuft, ins [src/](#)-Modul. Die Notebooks *importieren* nur — sie definieren keine eigenen Modelle mehr. So vermeidet man, dass Notebook und Automatik unterschiedliche Versionen rechnen.

Tor-Modell mit Dixon-Coles

Wir starten mit einem statistischen Klassiker: Tore sind seltene Ereignisse, also Poisson-verteilt — und Dixon-Coles erweitert das Modell um Stärke-Parameter pro Team plus eine Korrektur für niedrige Scores.

Poisson-Verteilung — die Grundidee

Wenn ein Team im Schnitt 1.4 Tore pro Spiel macht, dann ist die Wahrscheinlichkeit für genau k Tore gegeben durch:

$$P(k \text{ Tore}) = \lambda^k \cdot e^{-\lambda} / k!$$

λ (lambda) = erwartete Tore in diesem Spiel, $e \approx 2.718$, $k!$ = k-Fakultät

Wenn man die Tor-Erwartung für Heim- und Auswärtsteam unabhängig modelliert, bekommt man eine ganze *Score-Matrix*: $P(0:0)$, $P(1:0)$, $P(2:1)$, ... Summiert man die Felder unterhalb der Diagonale, ergibt das die Heimsieg-Wahrscheinlichkeit. Diagonale = Remis. Oberhalb = Auswärtssieg.

Dixon-Coles — die drei Erweiterungen

Reine Poisson reicht in der Praxis nicht. Dixon & Coles (1997) ergänzen drei Dinge:

1. **Stärke-Parameter pro Team**: Angriff α_i und Verteidigung β_i . Erwartete Tore Heim: $\lambda_H = \exp(\alpha_H + \beta_A + \gamma)$, wobei γ der Heimvorteil ist.
2. **Heimvorteil γ** als globaler Bonus — empirisch ungefähr 0.3 (multiplikativ etwa 1.35× mehr erwartete Tore zuhause).
3. **Korrektur τ für niedrige Scores**: Die unabhängige Poisson überschätzt 0:0, 1:1 und 1:0/0:1. τ korrigiert genau diese vier Zellen der Matrix nach unten bzw. oben.

Alle Parameter werden per **Maximum-Likelihood** aus den historischen Daten geschätzt — mit Zeitabschlag, sodass Spiele aus 1990 weniger zählen als solche aus 2024 (Halbwertszeit ungefähr 2 Jahre).

RPS — wie wir Modellgüte messen

Zu sagen "das Modell stimmt zu 60 %" reicht nicht — Wahrscheinlichkeiten sind reichhaltiger als Ja/Nein. Der **Ranked Probability Score** bewertet, wie nahe die prognostizierte Verteilung am tatsächlichen Ausgang lag. Niedriger = besser, perfekt = 0.

$$RPS = (1 / (n-1)) \cdot \sum_{i=1..n-1} (\sum_{j \leq i} (p_j - o_j))^2$$

$n = 3$ Ausgänge (Heim/Remis/Auswärts), p_j = prognostizierte W-keit, $o_j \in \{0, 1\}$ = tatsächlicher Ausgang

Das Quadrieren in den Klammern bestraft Modelle stärker, die falsch *und* selbstsicher waren. Der Baseline-Backtest auf der WM 2022 ergibt $RPS = 0.2316$.

CODE-POINTER

Das Modell wird in [notebooks/01_dixon_coles_baseline.ipynb](#) hergeleitet und auf der WM 2022 backtested. Die produktive Implementation liegt in [src/wm_model.py](#) in der Funktion `fit_goal_model()` und der Klasse `GoalModel`.

Monte-Carlo-Simulation des Turniers

Ein Modell pro Einzelspiel ist eine Sache. Die Frage "wer wird Weltmeister?" zu beantworten heisst, das ganze Turnier 10 000 Mal zu spielen — und zu zählen, wie oft jede Mannschaft am Ende den Pokal hochhebt.

Das WM-2026-Format

Erstmals 48 Teams, das macht die Simulation umfangreicher als früher:

- **Gruppenphase:** 12 Gruppen à 4 Teams, jeder gegen jeden — 6 Spiele pro Gruppe, gesamt **72 Spiele**
- **Aufstieg:** Top 2 jeder Gruppe (24 Teams) plus die **8 besten Gruppendritten** ergibt 32 Teams für die K.-o.-Runde
- **K.-o.-Phase:** Achtelfinale → Viertel → Halb → Final/Spiel um Platz 3 — **32 Spiele**
- **Total:** 104 Spiele über etwa 5 Wochen

So läuft eine einzelne Simulation

- 1 Für jedes der 72 Gruppen-Spiele: Tor-Modell ausrechnen, dann *aus der Verteilung ziehen* — d.h. ein konkretes Resultat würfeln. Beispiel: das Modell sagt 47/13/40 % Heim/Remis/Auswärts; im Würfel kommt 0.62 raus, also Auswärtssieg.
- 2 Tabellen pro Gruppe nach den FIFA-Tiebreaker-Regeln sortieren (Punkte → Tordifferenz → Tore → Direktvergleich).
- 3 Top 2 plus die 8 besten Dritten ermitteln. (Aktuell vereinfacht via Finish-Seeding, die exakte FIFA-Zuordnungstabelle folgt in Phase 2b der Roadmap.)
- 4 K.-o.-Bracket aufbauen, jedes Spiel wieder per Tor-Modell würfeln. Bei Remis nach 90 Minuten: zusätzlich Verlängerung & Penalties simulieren (vereinfacht 50/50).
- 5 Den Sieger des Finals notieren.

Dieses Spiel wiederholen wir 10 000-mal. Argentinien gewinnt in 1 400 dieser Universen? → Titel-Chance = 14 %.

Warum genau 10 000?

Bei 10 000 Iterationen ist der **Standardfehler** einer 10 %-Wahrscheinlichkeit etwa $\sqrt{(0.1 \cdot 0.9 / 10000)} \approx 0.3$ Prozentpunkte — präzise genug für Tagesvergleiche, schnell genug für den 06:00-Cron (etwa 8 Sekunden auf einem GitHub-Actions-Runner). Mit 100 000 Sims würde der Fehler nur um Faktor $\sqrt{10}$ kleiner — der Aufwand aber 10× grösser.

CODE-POINTER

[notebooks/02_monte_carlo_simulation.ipynb](#) erklärt das Format und visualisiert das Bracket. Produktiv: `wm.simulate_tournament()` in [src/wm_model.py](#). Aggregation zu Titel/Finale/Halbfinale-W-keit in `build_forecast()` .

Elo-Engine als Modell-Upgrade

Dixon-Coles funktioniert, hat aber Schwächen — vor allem für selten spielende Nationen ist die Schätzung instabil. Elo-Ratings sind robuster und kalibrieren sich nach jedem Spiel selbst.

Wie Elo funktioniert

Jedes Team hat eine Rating-Zahl R . Vor jedem Spiel berechnet man die erwartete "Punktzahl" (Sieg = 1, Remis = 0.5, Niederlage = 0):

$$E_A = 1 / (1 + 10^{(R_B - R_A) / 400})$$

E_A = erwartete Punkte für Team A · R_A , R_B = aktuelle Ratings · 400 = Schach-Standard-Skala

Nach dem Spiel wird das Rating updatet — je grösser die Überraschung, desto stärker:

$$R_{A, \text{neu}} = R_A + K \cdot (S_A - E_A)$$

$S_A \in \{0, 0.5, 1\}$ = tatsächliches Ergebnis · K = Lernrate (hier 30 für WMs, 20 für Freundschaftsspiele)

Vom Rating zur Sieg-Wahrscheinlichkeit

E_A aus der Formel oben ist die erwartete Punktzahl — also etwas zwischen 0 und 1. Mit einer Logistikk-Funktion und Daten-Fit übersetzt man das in Heim/Remis/Auswärts-Wahrscheinlichkeiten. Diese Übersetzung lernt das Modell ebenfalls aus den historischen Spielen (Logistic Regression auf RatingDifferenz $\rightarrow 1/X/2$).

Was bringt's konkret?

Backtest auf der WM 2022 (alle 64 Spiele):

MODELL	RPS (NIEDRIGER = BESSER)	Δ ZUR BASELINE
Dixon-Coles (Baseline)	0.2316	—
Elo-Engine	0.2215	-4.4 %

4.4 % klingt wenig — ist aber im Fussball-Prognose-Kontext substanziell. Profi-Bookmaker bewegen sich typischerweise bei RPS ~ 0.19 .

Heimvorteil bleibt drin

Beim Update gibt es einen Bonus, wenn das Heimteam tatsächlich zuhause spielt (nicht auf neutralem Boden). Bei WM-Spielen heisst "zuhause" nur: Spiel im eigenen Land — d.h. für die WM 2026 sind nur Spiele mit Mexico/USA/Canada als Heimteam echte Heimspiele.

CODE-POINTER

Herleitung mit Hyperparameter-Suche in [notebooks/03_elo_model.ipynb](#). Implementation als Klasse `EloEngine` in [src/wm_model.py](#) — bekommt einen historischen DataFrame, baut interne Rating-Tabelle, bietet `predict()` für eine Match-Paarung.

Live-Schicht mit TheSportsDB

Während das Turnier läuft, wollen wir gespielte Resultate ins Modell zurückspielen. Das passiert in zwei Schritten — Fixieren und Re-Simulieren.

Die Datenquelle

TheSportsDB liefert über die Endpoint `eventsseason.php?id=4429&s=2026` alle WM-Spiele (Liga-ID 4429 = FIFA World Cup) inklusive Score-Felder, sobald gespielt. Der API-Aufruf ist eine einzige HTTP-GET-Anfrage; bei Free-Tier liefert die WM-Liga keine Daten, deshalb wird ein Patreon-Key gebraucht. Der Key landet in der Umgebungsvariable `SPORTSDB_KEY` — lokal über die Shell, in der CI über ein Repo-Secret.

```
API_KEY = os.environ.get("SPORTSDB_KEY", "")
base = f"https://www.thesportsdb.com/api/v1/json/{API_KEY}"
r = requests.get(f"{base}/eventsseason.php",
                 params={"id": 4429, "s": "2026"}, timeout=20)
events = r.json().get("events") or []
```

Was im Modell passiert

- 1 **Live abrufen:** alle Spiele holen, die einen `intHomeScore` haben (also tatsächlich gespielt sind).
- 2 **Elo updaten:** jedes gespielte Spiel in chronologischer Reihenfolge ins Rating einfließen lassen.
- 3 **Spiele fixieren:** in der Monte-Carlo-Simulation werden gespielte Resultate *nicht* mehr gewürfelt, sondern als gegeben verwendet. So bleiben die simulierten Universen mit der Realität konsistent.
- 4 **Rest neu simulieren:** alles Übrige (offene Gruppenspiele, K.-o.-Bracket) wird wie gehabt 10 000-mal durchgespielt.

Robust gegen Probleme

Was passiert, wenn die API down ist, der Key fehlt oder ein Teamname nicht aufgelöst werden kann? Das Skript fängt jeden Fehler ab und fällt sauber auf die *Vorturnier-Prognose* zurück — also auf das Modell ohne Live-Updates. Eine Warnzeile im Log macht das transparent, aber der Cron-Lauf scheitert nicht.

NAME-MAPPING

TheSportsDB nennt Teams teils anders als der Trainingsdatensatz (z.B. "USA" statt "United States"). Das Mapping ist in [src/live_forecast.py](#) in der Tabelle `NAME_MAP` hartkodiert und wurde am 23.05.2026 gegen den vollständigen Spielplan verifiziert.

Pre-Match-Snapshots einfrieren

Damit das Dashboard später "Prognose vor Anpfiff vs. tatsächliches Resultat" zeigen kann, schreibt das Skript bei jedem Lauf für jedes anstehende Spiel eine eingefrorene Vorhersage in

[output/predictions_archive/YYYY-MM-DD.json](#). Sobald ein Spiel tatsächlich gespielt ist, fällt es aus der "offenen"-Liste raus, und der zuletzt geschriebene Snapshot bleibt als Referenz stehen.

Pro-Spiel-Prognose (1/X/2)

Manche Leser:innen wollen nicht "wer wird Weltmeister" wissen, sondern "wer gewinnt heute". Diese Sicht ist Ebene A direkt — die Bausteine, aus denen Monte Carlo die Titel-Wahrscheinlichkeit baut.

Was rauskommt

Pro noch nicht gespieltes WM-Spiel:

- **1/X/2** — Heimsieg / Remis / Auswärtssieg in Prozent
- **Wahrscheinlichstes Resultat** — die hellste Zelle der Score-Matrix (z.B. "2:1")
- **Wahrscheinlichkeit dieses Resultats** — meist nur etwa 9-13 %, weil viele Resultate ähnlich plausibel sind
- **Heimvorteil-Flag** — ist das ein echtes Heimspiel eines Gastgebers?

Notebook 05 ist die interaktive Variante mit Filtern: nach Team, nach Datum, oder "spannendste Spiele" (jene mit den ausgeglichensten 1/X/2-Verteilungen — die machen am meisten Spass anzuschauen).

CODE-POINTER

Funktion `predict_remaining_matches()` in [src/wm_model.py](#). Headless-Output unter [output/matches.json](#) bzw. [matches.md](#) für die Anzeige direkt im Repo.

Automatik via GitHub Actions

Damit niemand jeden Morgen ein Notebook starten muss, übernimmt GitHub Actions den Job — kostenlos, zuverlässig, und der commit kommt direkt vom Bot.

Was ist ein GitHub-Actions-Workflow?

Eine YAML-Datei unter `.github/workflows/`, die GitHub als Definition für eine ausführbare Pipeline interpretiert. Auslöser sind *Events* — Push, Pull Request, Cron-Zeitplan, oder manueller Klick. Die Pipeline läuft auf einem temporären Linux-Runner; alle Steps darin haben Zugriff aufs Repo (ausgecheckt via `actions/checkout`) und können beliebige Shell-Befehle ausführen.

Der Workflow im Detail

```
name: WM 2026 Live-Prognose

on:
  schedule:
    - cron: "0 6 * * *"          # täglich 06:00 UTC
  workflow_dispatch: {}         # plus manueller Trigger

jobs:
  forecast:
    runs-on: ubuntu-latest
    permissions:
      contents: write            # Bot darf commit + push
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - name: Prognose berechnen
        env:
          SPORTSDB_KEY: ${ secrets.SPORTSDB_KEY }
        run: python src/live_forecast.py
      - name: Output nach docs/data spiegeln
        run: |
          mkdir -p docs/data
          cp output/forecast.json output/matches.json output/results.json docs/data/
          cp output/history.csv docs/data/
      - name: Ergebnis committen
        run: |
          git config user.name "github-actions[bot]"
          git config user.email "github-actions[bot]@users.noreply.github.com"
          git add output/ docs/data/
          git commit -m "Live-Prognose $(date -u +%Y-%m-%d)" || echo "Keine Aenderung"
          git push
```

Cron-Syntax kurz erklärt

"0 6 * * *" besteht aus fünf Feldern: *Minute* · *Stunde* · *Tag* · *Monat* · *Wochentag*. 0 6 heisst "Minute 0 der 6. Stunde" — und die Sterne stehen für "jeden Tag/Monat/Wochentag". Also: täglich um 06:00 UTC = 08:00 Schweizer Sommerzeit. Davor sind die Spiele des Vortags zu Ende und die API hat die Resultate.

Secrets — wie der API-Key sicher reinkommt

Der TheSportsDB-Key darf nicht im Code stehen, sonst sieht ihn jeder. Lösung: *Repository Secrets*. Im GitHub-Repo unter **Settings** → **Secrets and variables** → **Actions** einen neuen Secret `SPORTSDB_KEY` mit dem Patreon-Key anlegen. Der Workflow liest ihn über `{{ secrets.SPORTSDB_KEY }}` und übergibt ihn als Umgebungsvariable an Python — er taucht in Logs nur als `***` auf.

Permissions — warum ein "write"-Token nötig ist

Standardmässig haben Actions nur Lesezugriff aufs Repo. Wir wollen aber zurückcommitten. Die Zeile `permissions: contents: write` hebt das Token auf Schreibrecht. Das ist sicherer als ein persönlicher PAT, weil das Token nur für die Dauer des Workflow-Laufs gilt und genau diese eine Berechtigung hat.

Was passiert, wenn nichts neu ist?

Die Zeile `git commit ... || echo "Keine Aenderung"` sorgt dafür, dass der Lauf nicht failed, wenn die Outputs identisch sind. Praktisch passiert das eher selten — schon das Update-Timestamp im JSON ist neu, also gibt's praktisch jeden Tag einen Commit.

BEOBACHTEN UND DEBUGGEN

Im GitHub-Repo unter **Actions**-Tab sieht man jeden Lauf, ob er grün/rot war, und kann sich jedes Step-Log live oder im Nachhinein ansehen. Auf "Re-run failed jobs" klicken startet den Workflow nochmal mit der gleichen Konfiguration.

Dashboard auf GitHub Pages

Das Endprodukt — eine Single-Page-Webapp, die die JSON-Artefakte direkt im Browser rendert. Kein Server, kein Login, einfach eine bookmarkbare URL.

So fließen die Daten



Der Workflow kopiert die JSON/CSV-Dateien aus `output/` nach `docs/data/`, damit GitHub Pages sie unter einem festen URL-Stamm ausliefert. Das Dashboard-JavaScript holt die vier Dateien per `fetch()` und rendert sechs Sektionen — Titel-Chancen, Verlaufschart, nächste Spiele, gespielte Spiele (mit Prognose-vs-Realitäts-Vergleich), Bewegungs-Spotlight.

Warum statisches HTML, nicht Streamlit?

Streamlit braucht einen laufenden Python-Server — also entweder Streamlit Cloud (separater Account) oder die eigene Maschine. Statisches HTML auf Pages ist *immer* verfügbar, ohne dass irgendwas läuft. Die JSON-Dateien sind die "Datenbank" — minimal, versioniert, frei abrufbar.

AKTIVIERUNG IN 30 SEKUNDEN

Repo → **Settings** → **Pages** → Source *Deploy from a branch* → Branch `main`, Folder `/docs` → *Save*. Nach 1-2 Minuten ist die Seite unter `https://<user>.github.io/<repo>/` live.

Reproduktion — bau es selbst nach

Wer das Projekt für die nächste WM (oder eine andere Liga) nachbauen will, braucht keine zwei Stunden. Hier ist die Checkliste.

- 1 **Repo forken oder neu anlegen.** Auf GitHub auf das Quell-Repo gehen und "Fork" klicken, oder das Projekt klonen und in ein eigenes leeres Repo pushen.

```
git clone https://github.com/<user>/World-Cup-2026.git
cd World-Cup-2026
```

- 2 **Conda-Umgebung anlegen.** Python 3.12, alle Abhängigkeiten aus [requirements.txt](#).

```
conda create -n wm2026 python=3.12 -y
conda activate wm2026
pip install -r requirements.txt
```

- 3 **TheSportsDB-Account einrichten.** Auf [thesportsdb.com](#) einen Patreon-Account abschliessen (ab ca. 3 USD/Monat) und den persönlichen API-Key kopieren.

- 4 **Lokal testen.** Key in die Shell setzen, Skript starten — sollte vier JSON-Dateien in [output/](#) erzeugen.

```
$env:SPORTSDB_KEY = "DEIN_PATREON_KEY"
python src/live_forecast.py
```

- 5 **Secret im Repo hinterlegen.** Auf GitHub: *Settings* → *Secrets and variables* → *Actions* → *New repository secret*. Name: `SPORTSDB_KEY`, Value: dein Key.

- 6 **Pages aktivieren.** *Settings* → *Pages* → *Source: Deploy from a branch* → *Branch:* `main`, *Folder:* `/docs` → *Save*. URL ist `https://<user>.github.io/<repo>/`.

- 7 **Ersten Cron-Lauf triggern.** Im *Actions*-Tab den Workflow "WM 2026 Live-Prognose" auswählen → *Run workflow*. Nach 1-2 Minuten ist [docs/data/](#) gefüllt und das Dashboard zeigt Daten.

- 8 **Anpassen.** Andere Liga? Liga-ID in [src/live_forecast.py](#) (Variable `WC_LEAGUE_ID`) ändern, ggf. Saison anpassen. Andere Visualisierung? [docs/index.html](#) editieren — alles in einer Datei.

HÄUFIGE STOLPERSTEINE

Repo muss **public** sein für gratis Pages. Workflow committet nur, wenn die Pipeline `contents: write` hat. Wenn die Dashboard-URL nur 404 zeigt, checke ob Pages auf `/docs` zeigt (nicht `/ (root)`).

Datei-Referenz

Eine Karte des Repos — jede Datei mit einem Satz Zweck.

Kerncode

PFAD	ZWECK
<code>src/wm_model.py</code>	Die ganze Modell-Engine: Elo-Klasse, Tor-Modell, Gruppen-Sortierung, Monte-Carlo, Pro-Spiel-Prognose. Eine Datei, eine Wahrheit — Notebooks und Automatik importieren dieselben Funktionen.
<code>src/live_forecast.py</code>	Headless-Skript für die Automatik. Holt Live-Resultate, ruft <code>wm_model</code> auf, schreibt Artefakte nach <code>output/</code> , archiviert Pre-Match-Snapshots, baut <code>results.json</code> .

Notebooks (zum Lesen / Forschen)

DATEI	PHASE	INHALT
<code>01_dixon_coles_baseline.ipynb</code>	1	Tor-Modell von Grund auf, WM-2022-Backtest, RPS = 0.2316
<code>02_monte_carlo_simulation.ipynb</code>	2	WM-2026-Format, Bracket, 10 000 Simulationen, Titel-Verteilung
<code>03_elo_model.ipynb</code>	4	Elo-Engine, Hyperparameter-Suche, Backtest-Vergleich, RPS −4.4 %
<code>04_live_update.ipynb</code>	3	Trockentest der Live-Logik mit der echten API, Erklärung der Pipeline
<code>05_match_predictions.ipynb</code>	5	Pro-Spiel-Prognose interaktiv, Filter, "spannendste Spiele"

Daten und Artefakte

PFAD	ZWECK
<code>data/results.csv</code>	Trainingsdaten — alle internationalen Spielresultate seit 1872 (~47k Zeilen). Wird beim ersten Lauf automatisch von einem GitHub-Mirror geladen.
<code>output/forecast.json</code>	Titel-/Finale-/Halbfinal-Wahrscheinlichkeiten pro Team. Quelle für die Dashboard-Sektion "Titel-Chancen".
<code>output/matches.json</code>	Pro-Spiel-Prognose für alle offenen WM-Partien. Quelle für "Nächste Spiele".
<code>output/results.json</code>	Gespielte Spiele + die Pre-Match-Prognose dazu. Quelle für "Prognose vs. Realität".
<code>output/history.csv</code>	Zeitreihe: eine Zeile je Team und Tag. Quelle für Verlaufschart und Bewegungs-Spotlight.
<code>output/predictions_archive/</code>	Pro Spieltag eine JSON-Datei mit dem zuletzt eingefrorenen Forecast.
<code>output/goal_coefs.json</code>	Gecachte Modell-Koeffizienten — vermeidet, dass jeder Cron-Lauf neu fittet.
<code>output/*.md</code>	Markdown-Spiegel der JSONs — direkt im Repo lesbar, ohne Dashboard.

Frontend und Doku

PFAD	ZWECK
<code>docs/index.html</code>	Das Dashboard. Single-File, Vanilla JS, Chart.js via CDN, lädt die JSON-Dateien aus <code>docs/data/</code> .
<code>docs/data/</code>	Spiegel von <code>output/</code> für GitHub Pages. Wird vom Cron-Lauf befüllt.
<code>docs/methodik.md</code>	Mathematische Vertiefung — Poisson-Herleitung, Dixon-Coles-Original-Paper-Referenzen, RPS-Theorie.
<code>docs/anleitung.html</code>	Diese Anleitung (HTML-Quelle für das PDF).
<code>README.md</code>	Kurzfassung: Setup, Roadmap, Verweis auf alle anderen Doku-Dateien.

Infrastruktur

PFAD	ZWECK
<code>.github/workflows/forecast.yml</code>	Der Cron-Workflow. Läuft täglich 06:00 UTC oder per Hand. Holt Daten, rechnet, committet zurück.
<code>requirements.txt</code>	Python-Abhängigkeiten — minimal: pandas, numpy, scipy, requests.
<code>.gitignore</code>	Ignoriert <code>data/*.csv</code> (gross, wird zur Laufzeit geladen) und Standard-Python-Artefakte.

Schlusswort

Wer alles bis hier gelesen hat, kennt jeden beweglichen Teil des Projekts. Das Schöne an der Architektur: keine Komponente ist gross. Modell = etwa 400 Zeilen Python. Automation = 35 Zeilen YAML. Dashboard = eine HTML-Datei. Wenn du etwas anpassen willst — irgendetwas — weisst du jetzt, wo du suchen musst.